

Guide d'animation

Atelier · Je regarde, je choisis, j'enregistre — document réservé à la formatrice

Sommaire

Avant de lire	1
L'atelier en bref	3
Pour qui, pour quoi	3
La méthode, en trois règles	3
L'histoire qu'on raconte	3
Les trois images qui portent tout	3
Le minutage d'ensemble	4
Ce que tu dois savoir toi-même	5
Les commandes de la soirée	5
Les mots à connaître	5
Les quatre phrases anglaises de Git	5
Deux commandes à ne pas montrer	6
Préparer l'atelier	7
J-7 : tester pour de vrai	7
J-3 : le mail	7
La veille	7
La dernière demi-heure	7
Le déroulé minute par minute	8
Ouverture (0:00 → 0:12)	8
Temps 1 — Une version, c'est quoi ? (0:12 → 0:27)	8
Temps 2 — Le geste (0:27 → 0:55)	9
Quiz éclair (0:55 → 0:58)	11
Pause (0:58 → 1:03)	12
Temps 3 — Vendredi, ça marchait (1:03 → 1:28)	12
Temps 4 — Ce qui a sa place dans le dépôt (1:28 → 1:36)	14
Temps 5 — Partager sur GitLab (1:36 → 1:52)	15
Clôture (1:52 → 2:00)	16
Corrigés	17
Le vote du Temps 1	17
Le quiz éclair	17
L'enquête du Temps 3	17
Le tri de cartes du Temps 4	17
Pratique ou démo : le récapitulatif	19
Quand ça coince	20
Les messages de Git à ne pas craindre	20
Plan B	20
Annexes	21
Les empreintes complètes de Marie	21
D'où viennent les sorties affichées	21
Si tu modifies les supports	21

Avant de lire

Ce guide te permet d'animer l'atelier **sans rien connaître d'autre** que lui. Tu peux le lire d'une traite la veille (45 minutes), puis le garder imprimé à côté de toi le jour J.

Il suit l'ordre de la soirée :

1. **L'atelier en bref** : à quoi il sert, l'histoire qu'on raconte, les trois images qui portent tout.
2. **Ce que tu dois savoir toi-même** : les commandes de la soirée, expliquées pour toi.

3. **Préparer** : J-7, J-3, la veille, la dernière demi-heure.
4. **Le déroulé minute par minute** : pour chaque moment, ce que tu fais, ce que tu dis, ce que les participants tapent et ce qu'ils doivent voir.
5. **Corrigés, dépannage, plan B** et les annexes.

Les conventions de lecture :

- 🕒 **0:27 → 0:29** : l'horloge de l'atelier (0:00 = le début), et la slide concernée entre guillemets.
- Un **encadré coloré** contient une phrase à dire à voix haute. Dis-la avec tes mots, mais garde l'idée.
- **Ils tapent** : les commandes que les participants tapent dans leur terminal. **Ils voient** : ce que Git doit leur répondre.
- **Pratique / Démo** : chaque exercice a deux versions. La pratique (tout le monde tape) est la version par défaut ; la démo (tu tapes, ils prédisent) sert si le groupe décroche ou si tu es en retard.

Ce que tu as sous la main :

Support	Où	À quoi il sert
Les slides	slides.html sur le site	Ce qui est projeté. Touche S : les notes (le même discours que ce guide, en plus court) ; touche F : plein écran
Le guide participant	atelier.html sur le site	Ce que les participants ont ouvert : toutes les commandes, avec les sorties attendues
Le kit	kit/kit.sh , lancé par chacun avant l'atelier	Crée les deux dossiers d'exercice (voir plus bas)
Le bac à sable	bac-a-sable.html sur le site	La même étude dans le navigateur, sans Onyxia : ton plan B
Le mémo et les cartes	materiel/memo.pdf , materiel/cartes-tri.pdf	À imprimer

L'atelier en bref

Pour qui, pour quoi

Le public : des statisticiennes et statisticiens, des data scientists, qui codent en R ou en Python dans VS Code sur Onyxia. La plupart n'ont **jamais utilisé Git**, ou l'ont subi. Ils connaissent très bien le problème (les fichiers `rapport_final_v2_corrige.docx`), pas la solution.

La promesse : en deux heures, chacun aura fait sur son propre poste trois choses :

1. **photographier** l'état exact de son étude, avec le pourquoi (`git add`, `git commit`) ;
2. **retrouver** une version passée (`git log`, `git show`, `git restore`) ;
3. **partager** son travail sur GitLab (`git push`).

Ce que l'atelier ne fait pas : les branches, la fusion, le travail à plusieurs (`pull`, `clone`, merge requests). Ce n'est pas l'objet de cet atelier.

La méthode, en trois règles

1. **Une seule étude du début à la fin**. On ne change jamais d'exemple : tout se passe dans l'étude fictive `entreprises_defense`.
2. **Toujours le même rythme** pour chaque notion : un **problème** concret, une **image** simple (l'album photo), le « **sous le capot** » (ce que Git fait vraiment), puis « **à vous** » (ils tapent).
3. **On prédit avant de taper**. En binôme, le copilote dit à voix haute ce que `git status` va répondre avant que le pilote appuie sur Entrée. Ce réflexe est l'apprentissage principal de la soirée.

L'histoire qu'on raconte

Marie, statisticienne, mène l'étude `entreprises_defense` : 40 entreprises inventées (SIREN commençant par 000), dont on calcule la dépendance à la défense. **Franck** est son collègue. Tout est synthétique.

Le kit, lancé par chaque participant avant l'atelier, crée deux dossiers dans `~/work/atelier-git/` :

Dossier	Ce qu'il contient	Quand on s'en sert
<code>entreprises_defense/</code>	L'étude sans Git (pas de dossier <code>.git</code>). C'est le dossier de chaque participant	Temps 2, 4 et 5
<code>entreprises_defense_marie/</code>	La même étude, avec l'historique de Marie : 8 commits datés de septembre 2026	Temps 3 et 4

L'historique de Marie raconte une petite enquête. Retiens surtout les commits 6 et 7 :

#	Date	Qui	Ce qu'il fait
1 à 5	1er au 10 sept.	Marie, puis Franck	L'étude se construit : import, filtre à 20 %, tailles PME/ETI/GE, correction des doublons
6	ven. 11 sept.	Marie	Le graphique passe en pourcentages , « à la demande du chef de bureau ». Le script marche : c'est « la version de vendredi »
7	lun. 14 sept.	Marie	Ajoute une ventilation par secteur... avec une colonne secteur qui n'existe pas . Le script plante
8	mer. 16 sept.	Marie	Met à jour le rapport

Le scénario du Temps 3 : Marie est en congé. Franck ouvre l'étude : le graphique a changé, personne ne sait pourquoi, et le script plante. Grâce à l'historique, les participants retrouvent qui a changé le graphique et pourquoi (commit 6), trouvent le commit qui casse (commit 7), puis reviennent à la version de vendredi.

La panne est voulue. Dans `entreprises_defense_marie/`, `scripts/03_analyse.R` et `rapport.qmd` s'arrêtent sur `Column `secteur` is not found`. Ne la corrige jamais avant l'atelier : c'est l'exercice.

Chez Marie traînent aussi des fichiers jamais confiés à Git (un jeton dans `.Renviron`, un fichier Excel confidentiel, un parquet de 600 Mo...). Ils servent au Temps 4 : « qu'est-ce qui a sa place dans le dépôt ? ».

Les trois images qui portent tout

Chaque notion technique s'appuie sur une image. Garde-les tout l'atelier, et dis toujours leurs limites.

L'image	Ce qu'elle explique	Sa limite, à dire
L'album photo : chaque commit est une photo de toute l'étude, collée dans un album, avec une légende au dos (qui, quand, pourquoi)	Le commit, l'historique, <code>git log</code> (feuilleter l'album)	Un album papier grossirait à chaque photo ; Git ne recopie pas ce qui n'a pas changé
La table de sélection : avant de coller des photos, on les pose sur une table pour choisir lesquelles entrent dans l'album	<code>git add</code> (poser sur la table) puis <code>git commit</code> (coller toute la table)	Poser sur la table, c'est faire une copie de la version du moment, pas déplacer le fichier
L'exemplaire déposé au bureau : l'album reste chez toi, GitLab en garde un exemplaire que tout le monde peut feuilleter	<code>git push</code> , la différence Git / GitLab	Ce n'est pas une photocopie figée : on y ajoute des pages à chaque <code>push</code>

Le schéma des **trois zones**, que tu montreras au Temps 2, résume la table de sélection :

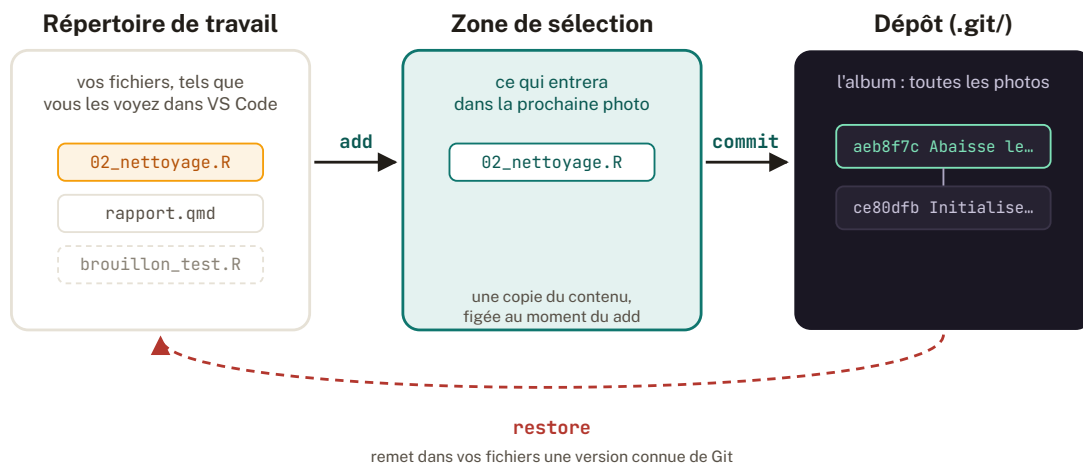


Figure 1. – Les trois zones de Git : le répertoire de travail (le bureau), la zone de sélection (la table), le dépôt (l'album).

Le minutage d'ensemble

Deux heures pile. Le détail minute par minute est dans le déroulé ; voici la vue d'ensemble.

Horloge	Durée	Séquence	En une phrase
0:00	12 min	Ouverture	Le film, les pires noms de fichiers, les trois objectifs
0:12	15 min	Temps 1 — Une version, c'est quoi ?	Un commit est une photo complète, avec une légende et un lien vers la précédente
0:27	28 min	Temps 2 — Le geste	<code>status</code> , <code>add</code> , <code>commit</code> : leurs deux premiers commits
0:55	3 min	Quiz éclair	Quatre questions de raisonnement
0:58	5 min	Pause	Tu vérifies les dossiers de Marie
1:03	25 min	Temps 3 — Vendredi, ça marchait	L'enquête dans l'historique de Marie, puis <code>git restore</code>
1:28	8 min	Temps 4 — Ce qui a sa place dans le dépôt	Tri de cartes, <code>.gitignore</code>
1:36	16 min	Temps 5 — Partager sur GitLab	Créer un projet, <code>git push</code>
1:52	8 min	Clôture	Le mémo, le ticket de sortie, le défi de la semaine

Si tu prends du retard, deux « fusibles », dans cet ordre :

- Temps 4** : saute le tri de cartes, donne directement le `.gitignore` et passe au Temps 5. Tu gagnes 5 minutes.
- Temps 3** : le `git restore --source` passe en démonstration sur ton écran au lieu d'être tapé par tous. Tu gagnes 2 à 3 minutes.

Ne sacrifie jamais le Temps 5 : c'est le seul moment où leur travail sort de leur service Onyxia, qui peut disparaître.

Ce que tu dois savoir toi-même

Tu n'as pas besoin d'être experte de Git. Il te faut comprendre une dizaine de commandes et les quatre phrases anglaises que Git affiche.

Les commandes de la soirée

Commande	Ce qu'elle fait	Ce qu'elle ne fait pas
<code>git init</code>	Crée l'album vide : le dossier caché <code>.git/</code> dans le dossier courant	Elle ne photographie rien
<code>git status</code>	Dit où en est chaque fichier : non suivi, modifié, sélectionné. Ne modifie jamais rien	Elle ne parle pas de GitLab, seulement du poste
<code>git add <fichier></code>	Pose la version actuelle du fichier sur la table de sélection	Si on modifie le fichier ensuite, la nouvelle version n'est pas sélectionnée : il faut refaire <code>add</code>
<code>git commit -m "..."</code>	Colle tout ce qui est sur la table comme une nouvelle photo de l'album, avec la légende	Sans <code>add</code> préalable, il n'y a rien à photographier : Git refuse
<code>git diff/git diff --staged</code>	Montre les lignes changées : entre le fichier et la table (<code>diff</code>), ou entre la table et la dernière photo (<code>--staged</code>)	Rien n'est stocké : le diff est calculé à la demande
<code>git log --oneline</code>	Feuilleter l'album, de la dernière photo à la première, une ligne par commit	Il ne montre pas le contenu des changements
<code>git show <empreinte></code>	Ouvre une photo : la légende complète et ce qui a changé par rapport à la précédente	Il ne modifie rien
<code>git restore <fichier></code>	Remet le fichier comme dans la dernière photo. Avec <code>--source=<empreinte></code> , comme dans une photo plus ancienne	Pas de corbeille : une modification jamais confiée à Git est perdue
<code>git push -u origin main</code>	Envoie sur GitLab les photos qu'il n'a pas encore. <code>-u</code> ne sert qu'au premier envoi	Il ne change rien sur le poste

Et pour le Temps 5 : `git remote add origin <adresse>` enregistre l'adresse du projet GitLab sous le surnom `origin`.

Les mots à connaître

- **Commit** : une photo complète de l'étude, avec l'auteur, la date, le message et un lien vers la photo précédente (le « parent »).
- **Empreinte** (hash, SHA-1) : 40 caractères calculés sur tout le commit, par exemple `155ae9a6e9d9...`. Les **7 premiers** suffisent pour désigner un commit (`155ae9a`). Changer un seul caractère du commit change toute l'empreinte : un commit ne se modifie pas.
- **Les trois zones** : le **répertoire de travail** (les fichiers dans VS Code, le bureau), la **zone de sélection** (la table ; staging area ou index en anglais), le **dépôt** (le dossier `.git/`, l'album).
- **HEAD** → **main** : « vous êtes ici », le dernier commit de la branche `main`.
- **origin** : un simple surnom pour l'adresse du projet GitLab.
- **Le jeton** : le mot de passe de GitLab, qu'Onyxia place dans chaque service depuis Mon compte. C'est pour cela que `git push` ne demande rien.

Les quatre phrases anglaises de Git

Git dit...	... et cela veut dire
<code>nothing to commit, working tree clean</code>	Rien à photographier : tout est déjà dans la dernière photo
<code>Untracked files</code>	Des fichiers que Git ne suit pas encore
<code>Changes not staged for commit</code>	Modifié, mais pas encore posé sur la table
<code>Changes to be committed</code>	Sur la table : entrera dans la prochaine photo

Deux commandes à ne pas montrer

`git reset --hard` (efface sans filet) et `git checkout` (fait trop de choses différentes selon ses arguments).
Si on te les demande : « elles existent, on ne les utilise pas ce soir ». Une seule commande pour revenir en arrière ce soir : `git restore`.

Préparer l'atelier

J-7 : tester pour de vrai

- ☐ **Faire tout le parcours d'un participant** sur un service VS Code neuf : jeton GitLab dans Mon compte sur Onyxia, `bash kit.sh`, un commit, un `git push` vers un projet de test. Aucun mot de passe ne doit être demandé.
- ☐ **Regarder le résultat du kit** : dans l'explorateur de VS Code, `work/atelier-git/` contient `entreprises_defense/` et `entreprises_defense_marie/`. Rappel : chez Marie, le script et le rapport plantent exprès (`Column `secteur` is not found`) ; dans `entreprises_defense/`, ils fonctionnent.
- ☐ **Vérifier GitLab** : la branche par défaut des nouveaux projets est `main` ; le groupe de l'équipe existe et les participants peuvent y créer des projets (sinon, leur espace personnel) ; la visibilité privée ou interne est autorisée ; la case « Initialiser le dépôt avec un README » est-elle cochée par défaut ? (Si oui, il faudra insister au Temps 5.)
- ☐ **Préparer ton plan B** : pousse ton propre `entreprises_defense_marie` sur le GitLab interne. Tu le montreras si un push échoue.
- ☐ **Vérifier la salle** : le site s'ouvre depuis les postes ; YouTube est accessible (sinon, copie `video/le-nouveau.mp4` sur ton poste).
- ☐ **Si tu as modifié le kit** : `bash kit/test_kit.sh` doit afficher « Tout est vert ».

J-3 : le mail

Envoie le mail de la page Avant l'atelier (section « Pour la formatrice ») : vingt minutes de préparation et une demande, **leur pire nom de fichier**. Les réponses ouvriront l'atelier.

La veille

- ☐ **Remplacer les cinq exemples de la slide « Vos pires noms de fichiers »** par les noms reçus (dans `slides.qmd`), puis régénérer le site. Garde les exemples fournis si tu en reçois moins de cinq.
- ☐ **Imprimer** : le mémo A4 (un par personne), les cartes de tri (un jeu par groupe de 2 ou 3, découpées), les tickets de sortie (une demi-feuille par personne avec trois lignes : « Une chose que je sais faire / Une chose encore floue / La commande que je taperai demain »).
- ☐ **Préparer la simulation papier** : trois feuilles A4 titrées Bureau, Table de sélection, Album ; sept post-its, un par fichier de l'étude (`README.md`, `rapport.qmd`, `01_import.R`, `02_nettoyage.R`, `03_analyse.R`, `brouillon_test.R`, `data/...csv`) ; un stylo, une agrafeuse.
- ☐ **Relire ce guide**, surtout le Temps 2 (la simulation) et le Temps 3 (l'enquête).

La dernière demi-heure

- ☐ Ouvre sur ton poste : les slides (en plein écran, touche `F`), ton service VS Code avec **deux terminaux**, l'un dans `~/work/atelier-git/entreprises_defense`, l'autre dans `~/work/atelier-git/entreprises_defense_marie`, et GitLab.
- ☐ Lance la vidéo une fois pour vérifier le son, puis remets-la au début.
- ☐ Pose les trois feuilles et les post-its sur une table visible de tous, tous les post-its sur Bureau.
- ☐ Prépare un tableau ou un paperboard pour noter les scènes du film et les critères du Temps 4.

Le déroulé minute par minute

Ouverture (0:00 → 0:12)

Objectif : que chacun se reconnaisse dans le problème avant qu'on parle de Git.

 **0:00 → 0:01** · **slide de titre**. Accueille. Vérifie d'un coup d'œil que chacun a son service VS Code ouvert sur Onyxia. Pas de tour de table maintenant.

« Deux heures, une seule étude, et à la fin vous saurez retrouver n'importe quelle version de votre travail. On commence par un film. »

 **0:01 → 0:09** · « **Le Nouveau** ». Lance le film et **tais-toi** pendant 8 minutes. Sans YouTube : [video/le-nouveau.mp4](#) en plein écran.

 **0:09 → 0:11** · « **Qui s'est reconnu ?** ». Un mot par personne, pas plus. Ne commente pas : note au tableau les scènes citées. Tu y feras référence pendant l'atelier (« vous vous souvenez de la scène où... »). Si personne ne se lance, commence par une vraie histoire à toi, courte.

 **0:11** · « **Vos pires noms de fichiers** ». Fais apparaître les noms un par un (flèche droite) et laisse rire. Aucune explication.

« Ce soir, on remplace tout ça par une seule chose. »

 **0:11:30** · « **Ce soir, vous saurez** ». Les trois objectifs, annoncés comme un contrat.

« Dans deux heures, chacun de vous aura fait ces trois choses sur son propre poste : photographier son étude, retrouver la version de vendredi, la partager sur GitLab. »

 **0:12** · « **Cinq portes d'entrée** ». Lis le texte de la slide, sans le paraphraser (15 secondes). L'idée : on aborde chaque notion par un récit, un geste, un schéma, une preuve et un échange, et **tout le monde passe par toutes les portes**. Ne parle jamais de « styles d'apprentissage » : cette idée n'est pas soutenue par la recherche.

 **0:12** · « **Le fil rouge** ». Présente l'étude, Marie et Franck, et précise que tout est fictif. Les participants en Python ont les mêmes fichiers en `.py` ; les commandes Git sont identiques.

« Premier temps : une version, c'est quoi exactement ? »

Temps 1 — Une version, c'est quoi ? (0:12 → 0:27)

Objectif : que chacun sache ce que contient un commit **avant** de taper la moindre commande. On ne touche pas au clavier.

Ce que tu dois avoir compris : un commit n'est pas une liste de différences, c'est une **photo complète** de toute l'étude. Git ne recopie pas les fichiers inchangés parce qu'il range chaque contenu sous son empreinte : un contenu identique n'est stocké qu'une fois. Le diff, lui, est recalculé à la demande en comparant deux photos.

 **0:12 → 0:14** · « **Vendredi, mon script fonctionnait** ». Laisse la phrase seule à l'écran quelques secondes.

« Qu'est-ce qu'il vous faudrait, là, tout de suite ? »

Recueille deux ou trois réponses, puis fais apparaître la suite : l'état exact de l'étude vendredi, et savoir ce qui a changé, par qui et **pourquoi**. Insiste sur le pourquoi : un état exact sans explication ne suffit pas pour reprendre le travail de quelqu'un.

 **0:14 → 0:18** · « **À chaque commit, Git enregistre... (vote)**. Trois réponses : (a) les différences, (b) une copie complète, (c) les seuls fichiers modifiés. Le déroulé, dans cet ordre :

1. vote à main levée, sans discussion (compte les mains) ;
2. « Convincez votre voisin » : 1 minute en binôme ;
3. second vote ;
4. seulement alors, la réponse : **(b)**.

La plupart votent (a) ou (c) : c'est normal. Ne corrige pas tout de suite, les deux slides suivantes s'en chargent. Si quelqu'un objecte que Git compresse par différences : c'est vrai à l'empaquetage, mais c'est une optimisation de stockage invisible ; le modèle reste la photo complète.

 **0:18 → 0:19** · « **La photo de l'album** ». Pose l'image qui servira toute la soirée.

« Chaque commit est une photo de toute l'étude, pas d'un morceau. La légende au dos, c'est le message : qui, quand, pourquoi. »

Dis tout de suite la limite : un album papier grossirait à chaque photo, pas Git.

 **0:19 → 0:21** · « **Un commit, vraiment** » (sous le capot). C'est l'explication qui corrige le vote : instantané complet ; les contenus identiques sont stockés une seule fois ; le diff (a) est une vue calculée entre deux photos ; (c) décrit ce que Git stocke **de neuf**, pas ce que le commit **contient**.

 **0:21 → 0:23** · « **L'empreinte** ». 40 caractères calculés sur tout le commit. Deux conséquences :

- **unicité** : deux commits différents ont des empreintes différentes ;
- **intégrité** : modifier un commit change son empreinte, et comme chaque commit contient l'empreinte de son parent, toutes les suivantes changeraient. L'historique est scellé.

L'empreinte affichée (155ae9a...) est un vrai commit de Marie : on la retrouvera au Temps 3.

 **0:23 → 0:25** · « **La chaîne** » (trois slides). Les trois premiers commits de Marie apparaissent un par un. Le premier n'a pas de parent. Chaque suivant pointe vers le précédent : **la flèche va vers le passé**.

« Un historique, ce n'est pas une liste de dates : c'est une chaîne de photos reliées, chacune avec sa légende. »

 **0:25 → 0:27** · « **À retenir** ». Fais répéter la phrase par un participant, avec ses mots : un commit = une photo complète + un message + un lien vers la précédente. Deux minutes de questions.

« Maintenant, le geste. On passe au terminal. »

Temps 2 — Le geste (0:27 → 0:55)

Objectif : chacun fait ses deux premiers commits en comprenant les trois zones. C'est **le cœur de l'atelier**.

Dossier : chacun ouvre le terminal intégré de VS Code (Terminal → Nouveau terminal) et tape :

```
cd ~/work/atelier-git/entreprises_defense
```

Ce que tu dois avoir compris : entre le fichier et l'album, il y a une étape, la **zone de sélection**. `git add` y pose une **copie de la version du moment** ; `git commit` photographie toute la sélection. Si on modifie le fichier après le `add`, la modification n'est pas sélectionnée. Tout le Temps 2 et la moitié du quiz reposent sur ce point.

 **0:27 → 0:29** · « **Qu'est-ce que je photographie ?** ». Tu fais une **démonstration** sur ton écran (ils la refont juste après) : `git init`, puis `git status`.

- `git init` crée le dossier caché `.git/` : l'album, vide.
- `git status` liste ce que Git voit mais ne suit pas encore.

« Qu'est-ce qui mérite la photo ? Tout ? Le brouillon aussi ? Les données ? »

Ne tranche pas : on y revient à l'étape 1 et au Temps 4.

 **0:29 → 0:30** · « **Git vous parle en anglais** ». Les quatre phrases du tableau de la partie « Ce que tu dois savoir ». Elles couvrent presque tout ce qu'ils liront ce soir.

 **0:30 → 0:31** · « **La table de sélection** ». L'image.

« Avant de coller les photos dans l'album, on les pose sur une table pour choisir. On peut y mettre le script de nettoyage et laisser le brouillon de côté. »

🕒 0:31 → 0:33 · « **Trois zones** ». Le schéma : le **répertoire de travail** (vos fichiers dans VS Code), la **zone de sélection** (ce qui entrera dans la prochaine photo ; c'est le seul moment où tu prononces le mot anglais staging area), le **dépôt** (le dossier `.git/`, l'album). `add` va de la première à la deuxième, `commit` de la deuxième à la troisième ; `restore` fait le chemin inverse (Temps 3).

🕒 0:33 → 0:34 · « `git add` fige une version ».

« `git add` ne sélectionne pas un fichier : il sélectionne une version d'un fichier. Retenez-le, ça va servir dans dix minutes. »

🕒 0:34 → 0:39 · « **Simulation : Git, c'est moi** ». Tu joues Git avec les trois feuilles et les post-its. Tu n'agis **que sur commande** : ce sont les participants qui disent les commandes à voix haute. Tous les post-its commencent sur Bureau.

1. « `git status` » → tu lis : « Je vois sept fichiers que je ne suis pas. »
2. « `git add README.md` » → tu **recopies** le post-it sur Table de sélection. L'original reste sur Bureau : c'est une copie, pas un déplacement.
3. Un participant gribouille le post-it `README.md` resté sur Bureau. « `git status` » → « `README.md` est sur la table, et il a changé sur le bureau depuis. » Montre les deux versions différentes.
4. « `git commit -m "..."` » → tu prends tout ce qui est sur la table, plus la dernière version de ce qui était déjà dans l'album, et tu agrafes le tout comme une nouvelle page d'Album, avec la légende. La table se vide.
5. « `git commit` » sans rien sur la table → « Rien à photographier. »

Ce que la simulation montre : `add` copie, `commit` prend toute la sélection, une page d'album est toujours complète.

« Maintenant, pour de vrai. En binômes. »

🕒 0:39 → 0:45 · « **Étape 1 : la première photo** ». En binômes : le **pilote** tape, le **copilote** prédit à voix haute ce que `git status` va répondre, **avant** que le pilote appuie sur Entrée. La prédiction est le vrai exercice.

Ils tapent :

```
git init
git status
git add README.md scripts/ rapport.qmd
git status
git commit -m "Initialise le suivi de l'étude entreprises_defense"
git status
```

Ils voient : après le `add`, deux listes. Changes to be committed contient les cinq fichiers sélectionnés (la table) ; Untracked files contient `brouillon_test.R` et `data/`. Le commit répond `[main (root-commit) ce80dfb] Initialise...` ; `root-commit` signale le premier commit, sans parent. Au dernier `git status`, seuls `brouillon_test.R` et `data/` restent dehors.

Fais remarquer au deuxième `git status` que le CSV et le brouillon restent dehors : **tout ne mérite pas la photo**.

« Pourquoi votre empreinte est-elle différente de la mienne ? »

Réponse : l'auteur et la date font partie de ce qui est résumé par l'empreinte (Temps 1).

Si l'éditeur s'ouvre en plein écran, avec des `~` à gauche : quelqu'un a oublié `-m`. Échap, puis `:q!`, puis Entrée, et on recommence avec `-m "..."`.

Démo : tu tapes, la salle prédit à voix haute.

🕒 0:45 → 0:50 · « **Étape 2 : changer le seuil** ». Les binômes **échangent les rôles**. Dans VS Code, ils ouvrent `scripts/02_nettoyage.R`, passent `seuil_dependance` ← `0.20` à `0.10`, et enregistrent (en Python : `seuil_dependance = 0.20` dans `scripts/02_nettoyage.py`).

Ils tapent :

```
git status
git diff
git add scripts/02_nettoyage.R
git diff --staged
git commit -m "Abaisse le seuil de dépendance défense à 10 %"
git log --oneline
```

Ils voient :

- `git status`: `modified: scripts/02_nettoyage.R` sous `Changes not staged for commit` ;
- `git diff` : une ligne `-seuil_dependance ← 0.20` (en rouge) et une ligne `+seuil_dependance ← 0.10` (en vert). Dis-leur d'ignorer les quatre premières lignes d'en-tête ;
- après le `add`, `git diff` ne montrerait plus rien, et `git diff --staged` montre ce qui entrera dans la photo ;
- `git log --oneline` : deux lignes, la plus récente en haut, avec `(HEAD → main)` (« vous êtes ici »).

Si un binôme est en avance : fais-lui taper `git diff` juste après le `git add`, et expliquer pourquoi c'est vide.

Démo : tu modifies le seuil sur ton écran, ils prédisent.

 **0:50 → 0:51** • « **La même chose, avec des cases à cocher** ». Ouvre le panneau Source Control de VS Code (icône de branche à gauche, ou `Ctrl+Maj+G`) sur ton écran. `Changes` = modifié mais pas choisi ; `Staged Changes` = la sélection ; le `+` à côté d'un fichier = `git add` ; un clic sur un fichier = le diff côte à côte.

« C'est pratique, mais la commande reste la référence. En cas de doute, `git status` dans le terminal dit la vérité. »

 **0:51 → 0:54** • **Les trois pièges (trois slides)**. On les **provoque exprès**, pour les reconnaître plus tard.

1. « **Commit sans add** » : ils modifient `README.md`, puis tapent `git commit -m "Ajoute une note de relecture"` sans `add`. Git refuse : la dernière ligne dit `no changes added to commit`. Faites-la lire ensemble : la table était vide.
2. « **Modifier après add** » : `git add README.md`, puis une nouvelle ligne dans `README.md`, puis `git status`. Le fichier apparaît **dans les deux listes** : une version sélectionnée, une plus récente. Deux sorties : refaire `git add` pour sélectionner la plus récente, ou retirer le fichier de la sélection avec `git restore --staged README.md` (Git le suggère lui-même). Pour nettoyer avant la suite : `git restore --staged README.md` puis `git restore README.md` (« on remet le fichier comme dans la dernière photo » ; on reverra `restore` au Temps 3).
3. « **Le message de commit** » : un message décrit l'**intention**, à l'impératif, pour la personne qui lira dans trois mois.

« Un bon message se lit comme la suite de « Ce commit... » : « Ce commit abaisse le seuil de dépendance défense à 10 % ». « modif » ne dira rien à personne dans trois mois. »

 **0:54 → 0:55** • « **À retenir** ». Regarder, choisir, enregistrer.

« `git status` ne casse jamais rien. Tapez-le sans arrêt : c'est la réponse à toute hésitation. »

Quiz éclair (0:55 → 0:58)

Objectif : vérifier qu'ils raisonnent avec les trois zones, pas qu'ils récitent. Pour chaque question, 45 secondes : 15 secondes de réflexion, une réponse à voix haute, puis la réponse à l'écran. Les réponses détaillées sont dans les **Corrigés**.

1. On a modifié deux fichiers, sélectionné un seul, puis commité : que contient la photo ? → **Tout le projet**, avec l'ancienne version du fichier non sélectionné.
2. `git add`, une correction, puis `git commit` : la correction est-elle enregistrée ? → **Non**. Si la salle hésite, retourne un instant au schéma des trois zones.
3. `nothing to commit, working tree clean` : l'étude est-elle sur GitLab ? → **Non** : « photographié ne veut pas dire partagé ». Cela prépare le Temps 5.
4. Deux commits peuvent-ils avoir la même empreinte ? → **En pratique, non**.

Pause (0:58 → 1:03)

L'heure de reprise s'affiche sur la slide. **Pendant la pause**, fais le tour : chacun doit avoir le dossier `~/work/atelier-git/entreprises_defense_marie`. S'il manque, fais créer un second kit à côté, sans toucher au premier :

```
bash kit.sh --dest ~/work/atelier-git-2
```

La personne fera le Temps 3 dans `~/work/atelier-git-2/entreprises_defense_marie`, puis reviendra dans son dossier habituel au Temps 4. **N'utilise pas --force ici** : il recréerait aussi `entreprises_defense/` et effacerait les commits du Temps 2.

Temps 3 — Vendredi, ça marchait (1:03 → 1:28)

Objectif : retrouver dans un historique **qui** a changé quoi et **pourquoi**, puis revenir à une version passée sans effacer l'histoire.

Dossier : tout le monde passe dans l'étude de Marie.

```
cd ~/work/atelier-git/entreprises_defense_marie
```

Ce que tu dois avoir compris : `git log` remonte la chaîne des photos à partir de la dernière. `git show` ouvre une photo et la compare à la précédente. Pour revenir à la version de vendredi d'un fichier, on ne réécrit pas l'histoire : on reprend l'ancienne version avec `git restore --source=...`, puis on fait **un nouveau commit** qui explique le retour. Le commit fautif reste visible dans l'album.

Les empreintes de Marie sont **les mêmes sur tous les postes** (le kit fixe les dates et les auteurs) : tout le monde peut comparer ses réponses. Attention aux participants en Python : leurs empreintes sont différentes (voir l'annexe).

 **1:03 → 1:05** · « **Marie est absente** ». Raconte la scène.

« Marie est en congé. Le chef de bureau veut la note pour jeudi. Franck ouvre le dossier : le graphique n'est plus le même que la dernière fois, personne ne sait pourquoi. Et le script plante. »

Si R est disponible sur ton service, montre l'erreur en vrai : `Rscript scripts/03_analyse.R` s'arrête sur `Column `secteur` is not found`. Sinon, raconte-la.

 **1:05 → 1:06** · « **Feuilleter l'album** ». On part de la dernière photo, on remonte en lisant les légendes ; quand une légende intéresse, on regarde la photo de près.

 **1:06 → 1:08** · « **Remonter la chaîne** ». `git log` part de `HEAD` → `main` (« vous êtes ici ») et suit les liens vers les parents. `git show` compare une photo à sa parente : le diff est recalculé, il n'est stocké nulle part. Sept caractères de l'empreinte suffisent.

 **1:08 → 1:16** · « **L'enquête** » (binômes, 8 minutes). Deux questions : **qui a fait passer le graphique en pourcentages, quand, et pourquoi ?** Et **quel commit a cassé le script ?**

Ils tapent :

```
git log --oneline
git show <empreinte>
```

Ils voient : `git log --oneline` affiche les 8 commits de Marie, du plus récent (24e5e4a) au plus ancien (81bd623). La liste complète est en annexe.

Laisse chercher. Aide graduée, seulement si un binôme bloque :

1. « Lisez les messages. »
2. « Lequel parle de pourcentages ? Lequel parle de NAF ? »
3. « Tapez `git show` suivi des sept caractères. »

Si `git show` affiche : ou (END) en bas de l'écran : c'est le lecteur de texte de Git. Les flèches font défiler, q fait quitter.

Démo : tu cherches en direct, ils te guident.

 **1:16 → 1:17** · « **L'enquête : les réponses** ». Fais donner les réponses par les binômes **avant** de les afficher :

- le passage en pourcentages : `155ae9a`, Marie, vendredi 11 septembre à 17 h 20, « à la demande du chef de bureau ». C'est la version **de vendredi** ;
- le script cassé : `b0b1445`, lundi 14 septembre.

Montre ensuite `git show b0b1445` sur ton écran (slide « Le commit qui casse ») : cinq lignes ajoutées (les +), aucune retirée. La colonne `secteur` n'existe pas dans les données : il n'y a que `naf`. Marie voulait sans doute fabriquer un secteur à partir du code NAF, et ne l'a pas fait.

Si un binôme a ouvert `notes_reunion_chef_de_bureau.md` chez Marie : ces notes confirment l'histoire (le chef de bureau a demandé les parts ; la ventilation NAF était « à voir »).

 **1:17 → 1:18** · « **Et si le message avait été modif ?** ». Cette sortie est **fabriquée** pour l'exemple : dis-le.

« Combien de temps pour trouver le commit du chef de bureau, si tous les messages disaient « modif » ?
Il faudrait ouvrir les huit commits un par un. »

 **1:18 → 1:20** · « **Revenir : deux besoins, deux commandes** ».

- On a bidouillé un fichier et on veut revenir à la dernière photo : `git restore <fichier>`.
- On veut une version plus ancienne précise : `git restore --source=<empreinte> <fichier>`.

Ensuite, on vérifie (`git status`, `git diff`) et on enregistre ce retour par un **nouveau commit** qui explique le pourquoi.

« L'historique reste intact : le commit de lundi est toujours là. On ne réécrit pas le passé, on ajoute une page qui le corrige. »

 **1:20 → 1:21** · « **restore, vraiment** ». Sur le schéma des trois zones, `restore` est la flèche du retour, vers le répertoire de travail.

« Ce qui n'a jamais été ni `add` ni `commit` n'existe pas pour Git : `restore` l'écrase, et il n'y a pas de corbeille. »

 **1:21 → 1:23** · « **Revenir : la dernière photo** » (tout le monde). Ils modifient une ligne de `README.md` dans VS Code, enregistrent, puis :

```
git status
git restore README.md
git status
```

Ils voient : `modified: README.md`, puis plus rien de modifié. Dans VS Code, leur modification a disparu. C'est l'expérience à vivre : pas de corbeille.

 **1:23 → 1:26** · « **Revenir : la version de vendredi** ». Ils reprennent `03_analyse.R` tel qu'il était au commit de vendredi :

```
git restore --source=155ae9a scripts/03_analyse.R
git status
git diff
git add scripts/03_analyse.R
git commit -m "Revient au script d'analyse de vendredi (la ventilation NAF cassait le script)"
```

Ils voient : `git diff` montre les cinq lignes de la ventilation NAF **en -** (retirées) ; le commit répond `1 file changed, 5 deletions(-)`. Le script fonctionne de nouveau.

En Python : le fichier est `scripts/03_analyse.py` et le commit de vendredi est `693c70a`.

Démo (fusible n° 2) : tu le fais sur ton écran, ils prédisent le `git status`.

Si quelqu'un demande « et si je veux voir **tout** le projet tel qu'il était vendredi ? » : démonstration de 2 minutes maximum, sur ton écran seulement. `git switch --detach 155ae9a`, on regarde les fichiers dans VS Code, puis `git switch main` pour revenir. Ne le fais pas taper.

🕒 1:26 · « **Ce que l'on ne verra pas ce soir** ». `git reset --hard` et `git checkout` existent ; on ne les utilise pas ce soir. Elles ne sont pas sur la slide exprès : on ne montre pas ce qu'on ne veut pas voir taper.

🕒 1:26 → 1:28 · « **À retenir** ». Git ne peut retrouver que ce qu'on lui a confié.

« Le réflexe à emporter : avant de tenter quelque chose de risqué dans un script, faites un commit. Ensuite, vous pouvez tout essayer. »

« Chez Marie, `git status` liste des fichiers que personne n'a jamais regardés. »

Temps 4 — Ce qui a sa place dans le dépôt (1:28 → 1:36)

Objectif : savoir quoi confier à Git, et ne jamais y mettre de données ni de secrets.

Fusible n° 1 : si tu es en retard, saute le tri de cartes, montre directement la slide « Écrire un `.gitignore` » et passe au Temps 5.

Ce que tu dois avoir compris : Git garde chaque version de chaque fichier suivi, **pour toujours**. Un fichier de 600 Mo modifié dix fois, ce sont dix copies à garder et à télécharger par chaque collègue. Un secret commité reste dans l'historique, même si on supprime le fichier ensuite. Les données individuelles d'entreprises relèvent du secret statistique.

🕒 1:28 → 1:29 · « **J'ajoute tout ?** ». Toujours chez Marie, `git status` liste sept fichiers ou dossiers non suivis : `.Renviron`, `.Rhistory`, `data/bitd_2024.parquet`, `donnees_entreprises_confidentiel.xlsx`, `figures/`, `notes_reunion_chef_de_bureau.md`, `rapport.html`.

« On ajoute tout ? »

Laisse répondre, ne tranche pas : c'est l'objet du tri.

🕒 1:29 → 1:33 · « **Tri de cartes** » (groupes de 2 ou 3). Chaque groupe reçoit les 12 cartes et les range en trois piles : **à versionner**, **à ignorer**, **jamais**. Les cartes portent des précisions utiles : le CSV est un extrait synthétique de 40 lignes ; le parquet fait 600 Mo (celui du kit ne fait que 2 Mo) ; `.Renviron` contient un jeton ; le fichier Excel contient des données individuelles d'entreprises.

Pour le débrief, fais parler **les désaccords** d'abord. Trois cartes sont volontairement discutables : le CSV synthétique, le PNG et les notes de réunion. Le corrigé complet est dans les **Corrigés**.

Tri collectif (démo) : carte par carte, à main levée.

🕒 1:33 → 1:34 · « **Les critères** ». Écris au tableau les critères au fur et à mesure qu'ils sortent du débrief. Quatre questions suffisent à trier presque tout fichier :

1. **Texte ou binaire ?**
2. **Régénérable ou non ?**
3. **Secret ?** (un jeton, un mot de passe)
4. **Secret statistique ?** (des données individuelles)

🕒 1:34 · « **Ce que Git garde** ». Le lien avec le Temps 1 : un fichier modifié est un nouveau contenu, gardé pour toujours.

« Si un jeton se retrouve dans un commit, le supprimer dans le commit suivant ne sert à rien : il reste dans les photos précédentes. Il faut le révoquer immédiatement dans GitLab, puis en créer un autre. »

🕒 1:34 → 1:35 · « **Où range-t-on quoi ?** ». Le code et la documentation dans Git ; les données sur le stockage S3 d'Onyxia, en Parquet, lues par leur chemin dans les scripts ; les sorties (HTML, figures) régénérées à la demande.

Si quelqu'un demande comment sortir de Git un fichier déjà suivi : c'est hors programme ce soir (la commande est `git rm --cached`).

 **1:35 → 1:36** · « **Écrire un `.gitignore`** ». Chacun revient dans **son** dossier :

```
cd ~/work/atelier-git/entreprises_defense
git status
```

`data/` apparaît dans la liste. Dans VS Code : Fichier → Nouveau fichier, enregistré sous le nom `.gitignore` à la racine du dossier (le point au début compte), avec ce contenu :

```
data/
*.html
.Rhistory
.Renviron
figures/
```

Puis :

```
git status
git add .gitignore
git commit -m "Ignore les données, les sorties et les fichiers d'environnement"
```

Ils voient, au `git status` : `data/` a disparu ; il ne reste que `.gitignore` et `brouillon_test.R`. Le brouillon reste visible parce qu'on ne l'a pas ignoré : il est simplement non suivi (on peut en discuter).

« Tout ce qu'on a photographié ce soir est sur votre service Onyxia. Et s'il disparaît ? »

Temps 5 — Partager sur GitLab (1:36 → 1:52)

Objectif : chacun envoie son étude sur GitLab et y lit son historique.

Dossier : chacun reste dans `~/work/atelier-git/entreprises_defense`.

Ce que tu dois avoir compris : Git est l'outil, sur le poste. GitLab est un service qui **héberge une copie** du dépôt (mêmes commits, mêmes empreintes) et ajoute une interface web. `git push` envoie les commits que GitLab n'a pas encore. Le jeton qu'Onyxia a placé dans le service sert d'identité : aucun mot de passe n'est demandé.

 **1:36 → 1:37** · « **Et si le service disparaît ?** ». Le dossier `.git/` est dans le service : l'album disparaît avec lui. C'est la réponse complète à la question 3 du quiz.

 **1:37** · « **L'exemplaire déposé au bureau** ». L'image, et sa limite : ce n'est pas une photocopie figée, les deux albums se mettent à jour (on ajoute des pages avec `push`).

 **1:37 → 1:39** · « **Git ≠ GitLab** ». GitLab affiche l'historique, les diffs, le README mis en forme. Les empreintes sont les mêmes des deux côtés.

 **1:39 → 1:40** · « **origin, push, -u** ». `origin` n'est qu'un surnom pour l'adresse (c'est la convention). `-u` ne sert qu'au premier envoi : ensuite, `git push` suffit, et `git status` dit si on est à jour avec GitLab.

 **1:40 → 1:44** · « **Créer le projet dans GitLab** ». Fais-le en même temps qu'eux, sur ton écran :

1. **Nouveau projet** → **Créer un projet vide** (en anglais : New project → Create blank project) ;
2. nom : `entreprises-defense-⟨prénom⟩`, visibilité **privée** ou **interne** ;
3. **décocher** « Initialiser le dépôt avec un README ». Insiste lourdement : si la case reste cochée, le premier envoi sera refusé ;
4. copier l'adresse du projet, dans le bloc Push an existing folder. Ce bloc contient d'autres commandes : on n'en garde que l'adresse.

 **1:44 → 1:50** · « **Envoyer sur GitLab** ». Qu'ils **collent** l'adresse plutôt que de la taper :

```
git remote add origin <adresse copiée dans GitLab>
git push -u origin main
```

Ils voient : quelques lignes de progression (`Enumerating objects...`, `Writing objects...`), puis `* [new branch] main → main` et `branch 'main' set up to track 'origin/main'`. Les nombres d'objets varient d'un poste à l'autre.

Puis, dans GitLab : rafraîchir la page du projet ; ouvrir Code → Commits (ou Repository → Commits) ; cliquer sur un commit et lire son diff ; revenir à l'accueil du projet pour voir le README mis en forme.

« Comparez les empreintes dans GitLab et dans `git log --oneline` : ce sont les mêmes. C'est la même chaîne de photos. »

Démo : tu pousse ton projet ; ceux qui bloquent suivent sur ton écran.

 **1:50 → 1:51** · « Trois pièges ».

1. **Le projet a été créé avec un README** : le push est refusé (! `[rejected] ... (fetch first)`), car GitLab a un commit que le participant n'a pas. On ne dépanne pas ce soir : supprimer le projet dans GitLab, le recréer **sans README** avec le même nom, relancer `git push -u origin main`.
2. **master au lieu de main** : quelqu'un a fait `git init` avant de lancer le kit. `git push -u origin master` fonctionne aussi.
3. **Git demande un mot de passe** : le jeton est absent ou expiré. Vérifier Mon compte sur Onyxia, recréer le service. En attendant, **plan B** : montre ton projet `entreprises_defense_marie` déjà poussé, et la personne suit sur ton écran.

 **1:51 → 1:52** · « À retenir ». `commit` enregistre chez moi. `push` rend visible chez nous.

« Récapitulons. Treize lignes. »

Clôture (1:52 → 2:00)

 **1:52 → 1:54** · « Problème → commande ». Distribue le mémo imprimé. Parcours le tableau de haut en bas : chaque ligne est un moment de la soirée.

 **1:54 → 1:56** · « Ticket de sortie ». Papier, anonyme, ramassé à la sortie : une chose que je sais faire, une chose encore floue, la commande que je taperai demain. Garde-les : ils te disent quels « flous » reviennent le plus, pour y revenir avec le groupe après l'atelier si besoin.

 **1:56 → 1:58** · « Le défi de la semaine ». Mettre **une vraie étude** sous Git : trois commits avec de vrais messages, un push sur le GitLab interne. Propose-leur de t'envoyer le lien du projet.

 **1:58 → 2:00** · « Merci ». Rappelle que le bac à sable fonctionne dans le navigateur, sans Onyxia : pratique pour continuer à s'entraîner après l'atelier.

Corrigés

Le vote du Temps 1

(b) une copie complète du projet. Le commit est un instantané complet ; les fichiers inchangés ne sont pas recopiés parce que Git range chaque contenu sous son empreinte ; le diff (a) est calculé à la demande. Si quelqu'un objecte que Git compresse par différences : c'est vrai à l'emballage, mais c'est une optimisation de stockage invisible, le modèle reste la photo complète.

Le quiz éclair

1. **Que contient la photo ?** Tout le projet : la nouvelle version de `02_nettoyage.R`, et l'**ancienne** version de `rapport.qmd` (celle du commit précédent). La modification de `rapport.qmd` reste dans le répertoire de travail. Le piège : croire que la photo ne contient « que » le fichier sélectionné.
2. **La correction est-elle enregistrée ?** Non. `git add` a figé la version d'avant la correction ; `git status` montre le fichier dans les deux listes.
3. **L'étude est-elle à l'abri sur GitLab ?** Non. `git status` ne parle que du poste ; sans `git push`, GitLab n'a rien, et le service Onyxia peut disparaître.
4. **Deux commits peuvent-ils avoir la même empreinte ?** En pratique, non : l'empreinte porte sur tout le commit. Mathématiquement, des collisions SHA-1 existent (fabriquées exprès en 2017) ; Git détecte ces attaques et migre vers SHA-256. Par hasard, jamais.

L'enquête du Temps 3

Les empreintes de Marie, **produites par la version courante de `kit.sh` ; relancer `kit/test_kit.sh` après toute modification** (le test vérifie que les empreintes ci-dessous figurent toujours dans cette page) :

#	Date (2026)	Auteur	Message	Variante R	Variante Python
1	mar. 1 sept. 09:12	Marie	Initialise l'étude des entreprises liées à la défense	81bd623	c01700c
2	mer. 2 sept. 10:40	Marie	Ajoute l'import de l'extrait entreprises_defense_2024	3d6a872	d6a9b18
3	ven. 4 sept. 15:05	Marie	Ne garde que les entreprises dont la part de CA défense dépasse 20 %	af989d8	3948311
4	mar. 8 sept. 11:30	Marie	Ajoute la ventilation par taille d'entreprise (PME, ETI, GE)	842a418	b62bd95
5	jeu. 10 sept. 16:48	Franck	Corrige le calcul du CA défense (doublons de SIREN)	456a938	3c9fb14
6	ven. 11 sept. 17:20	Marie	Passe le graphique des effectifs en parts (%) à la demande du chef de bureau	155ae9a	693c70a
7	lun. 14 sept. 09:05	Marie	Ajoute la ventilation par secteur d'activité (NAF)	b0b1445	104a65d
8	mer. 16 sept. 14:10	Marie	Met à jour le rapport avec les résultats 2024	24e5e4a	c4a8e6b

Réponses attendues : le passage en parts, c'est le **commit 6** — Marie, vendredi 11 septembre à 17 h 20, « à la demande du chef de bureau ». Le script cassé, c'est le **commit 7** — lundi 14 septembre, `group_by(secteur)` alors que la colonne `secteur` n'existe pas (seule `naf` existe).

Si R est disponible sur ton service, tu peux montrer l'erreur en direct : `Rscript scripts/03_analyse.R` dans `entreprises_defense_marie/` échoue sur la colonne `secteur` ; après le `git restore --source`, il fonctionne. `kit/test_kit.sh` vérifie ce comportement (échec au commit 8, succès au commit 6) quand R est installé.

Le tri de cartes du Temps 4

Carte	Pile	Pourquoi
<code>scripts/02_nettoyage.R</code>	à versionner	Du code, du texte, le cœur de l'étude
<code>rapport.qmd</code>	à versionner	La source du rapport, du texte

Carte	Pile	Pourquoi
README.md	à versionner	La documentation
.gitignore	à versionner	Il se partage avec l'équipe
data/ entreprises_defense_2024.csv (extrait synthétique)	discutable	Petit, texte, non confidentiel : versionnable. Mais la règle générale reste : les données hors de Git
data/bitd_2024.parquet (600 Mo)	à ignorer	Binaire, volumineux, sa place est sur S3
rapport.html	à ignorer	Régénérable par <code>quarto render</code>
figures/ effectifs_par_taille.png	discutable	Régénérable par <code>03_analyse.R</code> : à ignorer, sauf s'il s'agit d'une pièce livrée à une date donnée
.Rhistory	à ignorer	Propre à chaque poste, sans valeur pour l'équipe
.Renviron (contient un jeton)	jamais	Secret : un jeton commité reste dans l'historique ; le révoquer aussitôt
donnees_entreprises_confidentiel.jamais	jamais	Données individuelles d'entreprises : secret statistique
notes_reunion_chef_de_bureau.md	discutable	Du texte utile, mais nominatif et interne : pas dans le dépôt de l'étude

Pratique ou démo : le récapitulatif

Moment	Version pratique (par défaut)	Version démo (si le groupe décroche ou si le temps manque)
Temps 2, étape 1	Binômes pilote/copilote, prédiction avant chaque <code>git status</code>	Tu tapes, la salle prédit à voix haute
Temps 2, étape 2	Les binômes échangent les rôles, même protocole	Tu modifies le seuil en partageant ton écran
Temps 2, pièges	Chacun provoque les trois pièges	Tu les provoques, ils lisent la sortie avec toi
Temps 3, enquête	Binômes, 8 min, aide graduée	Tu cherches en direct, ils te guident
Temps 3, <code>restore</code>	Tout le monde : <code>restore README.md</code> puis <code>--source</code>	<code>restore README.md</code> pour tous, <code>--source</code> en démo (fusible)
Temps 4, tri	Groupes de 2 ou 3, cartes papier	Tri collectif à main levée, carte par carte (ou fusible : <code>.gitignore</code> fourni)
Temps 4, <code>.gitignore</code>	Chacun crée le fichier dans VS Code	Tu le créés, ils le copient depuis le guide
Temps 5, push	Chacun crée son projet et pousse	Tu pousses ton projet ; ceux qui bloquent suivent sur ton écran

Quand ça coince

Les messages de Git à ne pas craindre

Ce qu'on voit	Ce que ça veut dire	Quoi faire
Un éditeur plein écran, des ~ sur la gauche	<code>git commit</code> sans <code>-m</code> : Git demande le message dans l'éditeur	Échap, puis <code>:q!</code> , puis Entrée ; recommencer avec <code>-m "..."</code>
<code>:</code> ou (END) en bas de l'écran après <code>git log</code> ou <code>git show</code>	Le lecteur de texte de Git (la sortie dépasse l'écran)	Flèches pour défiler, <code>q</code> pour quitter
<code>no changes added to commit</code>	Commit sans <code>add</code>	<code>git add <fichier></code> , puis <code>git commit</code>
<code>nothing added to commit but untracked files present</code>	Rien n'est sélectionné ; il reste des fichiers non suivis	Normal s'ils sont volontairement laissés dehors
<code>fatal: not a git repository</code>	On n'est pas dans le bon dossier	<code>cd ~/work/atelier-git/entreprises_defense</code> (ou <code>..._marie</code>)
<code>hint: Using 'master' as the name for the initial branch</code>	Le kit n'a pas été lancé avant <code>git init</code>	Pousser avec <code>git push -u origin master</code> , ou relancer le kit sur un service neuf
<code>! [rejected] ... (fetch first)</code>	Projet GitLab créé avec un README	Supprimer et recréer le projet sans README
Une demande de Username / Password au push	Jeton absent ou expiré	Vérifier Mon compte sur Onyxia, relancer un service ; sinon plan B
<code>Column `secteur` is not found</code> chez Marie	La panne voulue du commit 7	Rien : c'est l'exercice du Temps 3

Plan B

- **Onyxia indisponible** (panne, services qui ne démarrent pas) : tout l'atelier bascule sur le bac à sable du site (bac-a-sable.html), dans le navigateur. Chacun y ouvre la même étude que le kit, en R ou en Python. La prise en main couvre le Temps 2 ; les missions 1 à 3 le diff, le commit cohérent et le piège du `add` ; la mission 4 le `git restore` du Temps 3. Le `.gitignore` du Temps 4 y est visible (données et sorties ignorées), le tri de cartes se fait sur papier, le Temps 5 en démonstration sur ton écran.
- **Le push échoue** pour une partie de la salle : montre ton projet `entreprises_defense_marie` déjà poussé sur le GitLab interne (historique, un commit, son diff, le README) ; les personnes concernées suivent sur ton écran et referont le push chez elles (défi de la semaine).
- **Pas de YouTube dans la salle** : [video/le-nouveau.mp4](#) sur ton poste.
- **Service recréé le jour J** : `bash kit.sh --force` (le kit est idempotent : mêmes dossiers, mêmes empreintes).

Annexes

Les empreintes complètes de Marie

Variante R

```
81bd6237873727a79e027bf5c7e1d795b0a038b3 1 Initialise l'étude...
3d6a8722b4c203c6a1ba346894a8403567db5137 2 Ajoute l'import...
af989d85593715b68ab33fb55db0b453603e332c 3 Ne garde que...
842a418dadbc7a3df82204d7eaa546ffcdd55b28 4 Ajoute la ventilation par taille...
456a938d55720bc818d8dedf6d43d0734287e903 5 Corrige le calcul du CA défense...
155ae9a6e9d9be8237c6c7b5e152b26f5a022187 6 Passe le graphique... en parts (%)
b0b1445b986bec6278113229a4c5b64e30c17b89 7 Ajoute la ventilation par secteur (NAF)
24e5e4acd405c1b7c61ef0953d5e14629a4c5c48 8 Met à jour le rapport...
```

Variante Python

```
c01700c2175660f9babbbb766f109e183be86deb 1 Initialise l'étude...
d6a9b18c74d177019a5647e504f61163726b8d39 2 Ajoute l'import...
3948311611c9ca4f71b6847d1c5ae68c26b3e595 3 Ne garde que...
b62bd9527ef2073eb828adff5167ae73e439e683 4 Ajoute la ventilation par taille...
3c9fb1483f22a314cb114aec425d5134d91254e 5 Corrige le calcul du CA défense...
693c70a1e91dc3e6760af2ea89ba044061f2a63c 6 Passe le graphique... en parts (%)
104a65db0c1d6e2a8e6e5963e1af65515fbc712e 7 Ajoute la ventilation par secteur (NAF)
c4a8e6b67bc015a8a0f4108926b29c1cf60ffe4b 8 Met à jour le rapport...
```

D'où viennent les sorties affichées

Toutes les sorties des slides et du guide participant ont été produites en exécutant réellement les commandes sur le kit (Git 2.43, identité fictive « Camille », dates fixées pour la reproductibilité). Deux exceptions, signalées :

- la sortie de `git push` a été obtenue avec un dépôt distant local ; seule l'adresse a été remplacée par celle du GitLab interne ;
- la slide « Et si le message avait été modif ? » montre un historique fictif, fabriqué pour l'exemple, ce que tu dis à voix haute.

Les empreintes des commits des participants (Temps 2, 4 et 5) dépendent de leur nom et de l'heure : elles diffèrent d'un poste à l'autre. Celles de Marie sont identiques partout.

Si tu modifies les supports

Le discours de ce guide reprend les notes des slides (`slides.qmd`, blocs `notes`). Si tu changes le déroulé dans les slides, reporte-le ici, puis régénère le PDF : `cd formateur && quarto render`.